
MEMO: Nvidia CUDA programming get started

CUDA Programming Guide

- CPU/GPU をホスト/デバイスと呼び分けている
- CUDA API を通してホストメモリからデバイスメモリにデータをコピーすることができる
- Graphics Processing Clusters(GPCs) は Streaming Multiprocessors(SMs) の集まり
- SM は a local register file と a unified data cache と計算を行なう多数の機能ユニットを持つ
- 統合データキャッシュは共有メモリと L1 キャッシュの実体である
 - － 統合データキャッシュ → (共有メモリ, L1 キャッシュ) への割り当ては実行時に設定できる
- カーネル (GPU 上で実行される関数) はスレッドと呼ばれる単位で計算を行う (1 スレッドが 1CUDA コアに対応?)
 - － スレッドをまとめたものをスレッドブロックという
 - － スレッドブロックをまとめたものをグリッドという
 - * それぞれのまとめ方は 1,2,3 次元
 - * 同じグリッド内のブロックの形 (次元数) は同じ
 - － こうしてたくさん名称があるのは、処理したいデータ項目とスレッドを対応付けやすいため (実装の都合の、仮想的な概念か?)
- 同じスレッドブロックに存在するスレッドは同じ SM で実行される
 - － あるスレッドブロック内のスレッドは共有メモリを介してデータのやり取りが可能
 - － communicate や synchronize が有利
 - － 逆に、異なるスレッドブロックの間のデータのやり取りはオーバーヘッドが大きいことに注意すべし
- カーネルで実行される関数は必ず void

Listing 1 Kernel function

```
1  __global__ void vecAdd(float* A, float* B, float* C)
2  {
3
4  }
```

-
- カーネルでの計算結果は必ずカーネルに渡したポインタのアドレスに対する変更として使う
 - 実行構成 (execution configuration) で並列実行するスレッド数を指定できる

Listing 2 Execution Configuration

```
1    vecAdd<<1, 256>>(A, B, C);
```

参考

- Kazuki Fujii, CUDA Programming Guide Part 1
 - <https://zenn.dev/kaz20/articles/1e622ef249d133>
- Nvidia, CUDA Programming Guide
 - <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html>

Listing 3 How to build CUDA program

```
1 satsuki@satsuki-desktop:~/local/nvidia/cuda-samples/cpp/0_Introduction/  
  vectorAdd$ mkdir build  
2 satsuki@satsuki-desktop:~/local/nvidia/cuda-samples/cpp/0_Introduction/  
  vectorAdd$ cmake -S ./ -B ./build/ -DCMAKE_CUDA_ARCHITECTURES=86  
3 satsuki@satsuki-desktop:~/local/nvidia/cuda-samples/cpp/0_Introduction/  
  vectorAdd$ ./build/  
4 satsuki@satsuki-desktop:~/local/nvidia/cuda-samples/cpp/0_Introduction/  
  vectorAdd$ ./build/vectorAdd  
5 Unified Memory: CPU and GPU answers match
```

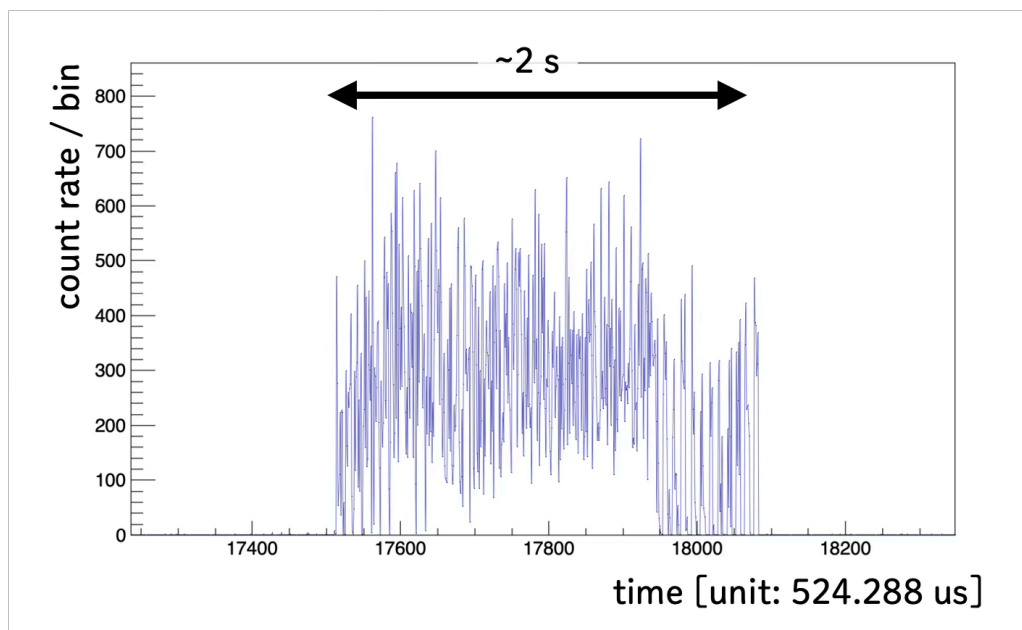


図 1 J-PARC ハドロン実験施設に供給されるビームの時間的な構造の例。約 2 秒の間、加速器施設のメインリングから徐々にビームを取り出す Slow extraction と呼ばれる方式でビームを検出器に照射している。横軸は時間で、縦軸は検出器のカウントレートである。